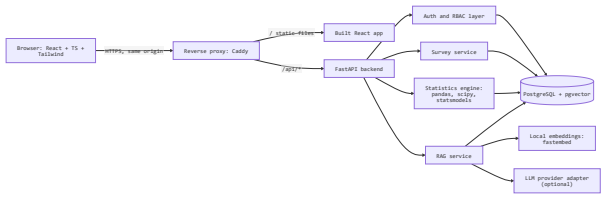
BankTech Analytics: Architecture and Implementation Blueprint

This document covers the architecture and the plan only. It contains no application code. When you're ready, ask for one module at a time (for example "Phase 2: backend auth"). I'll write those files, and Antigravity can apply them and run the tests.

A. Recommended architecture and justification

mermaid



Decision	Choice	Why
Shape	Modular monolith: one FastAPI app and one React SPA in one repo	Simplest to build, test and deploy for a student. Modules stay separate through folders, not separate services.
Same origin	Proxy `/api` through Vite in dev and through Caddy in production	No CORS or cross-site cookie problems, so HttpOnly session cookies just work.
DB access	SQLAlchemy 2.0 (sync) + psycopg 3 + Alembic	Sync code is easier to debug. FastAPI runs sync endpoints in a thread pool, which is fast enough for this scale.
Auth	Server-side sessions: opaque token in an HttpOnly cookie, hash stored in the DB	Logout and revocation actually work. No JWTs in localStorage, so XSS can't steal tokens. Easier to reason about than refresh-token rotation.
Vector store	pgvector inside the same PostgreSQL	One database. Access filtering happens in the same SQL query as retrieval, so restricted chunks can't leak.
Embeddings	`fastembed` with `BAAI/bge-small-en-v1.5` (384-dim, local ONNX model)	Free, offline and lightweight (no PyTorch). Tests don't need an API key.
LLM	Provider adapter: `none` (extractive), `openai`, `gemini`, `ollama`	The app works with no key. In `none` mode the assistant answers with quoted source passages.
Statistics	pandas + scipy + statsmodels, plus hand-written formula functions checked against them	Transparent formulas you can explain in your viva, backed by library cross-checks.

Decision	Choice	Why
Charts	Recharts	Simple React API, good enough for research dashboards.

B. Repository structure

```
banktech-analytics/
├── README.md
├── docker-compose.yml            # db (dev); full stack (prod profile)
├── .env.example                  # template only, never real secrets
├── .gitignore                    # includes .env, *.pem, uploads/
├── docs/
│   ├── architecture.md
│   ├── research-methodology.md   # hypotheses, constructs, formulas, limitations
│   ├── api.md                    # links to /api/docs (OpenAPI)
│   ├── deployment.md
│   └── handoff-log.md            # what each phase changed (for Antigravity)
├── backend/
│   ├── pyproject.toml
│   ├── alembic.ini
│   ├── alembic/versions/
│   ├── app/
│   │   ├── main.py               # app factory, routers, middleware
│   │   ├── cli.py                # create-admin, seed, ingest-docs, generate-synthe
│   │   ├── core/
│   │   │   ├── config.py         # pydantic-settings, reads env
│   │   │   ├── security.py       # hashing, token generation
│   │   │   ├── csrf.py
│   │   │   ├── permissions.py    # permission matrix (single source of truth)
│   │   │   ├── rate_limit.py
│   │   │   └── errors.py         # uniform error responses
│   │   ├── db/ (base.py, session.py)
│   │   ├── models/               # user.py, branch.py, session.py, invitation.py,
│   │   │                         # survey.py, response.py, analysis.py, document.py
│   │   ├── schemas/              # Pydantic request/response models
│   │   ├── api/
│   │   │   ├── deps.py           # get_db, get_current_user, require_permission
│   │   │   └── routes/           # health, auth, users, invitations, branches,
│   │   │                         # surveys, responses, analysis, dashboards,
│   │   │                         # documents, assistant
│   │   ├── services/
│   │   │   ├── auth_service.py
│   │   │   ├── survey_service.py
│   │   │   ├── stats/ (scoring.py, reliability.py, correlation.py,
│   │   │   │          regression.py, group_tests.py, report.py)
│   │   │   ├── rag/ (loader.py, chunker.py, embedder.py, retriever.py,
│   │   │   │        llm.py, citations.py, guardrails.py)
```

```
│   │   │   └── synthetic/generator.py
│   │   └── seed/ (surveys/*.json, faq_docs/*.md)
│   └── tests/ (conftest.py, unit/, integration/, security/, rag/)
└── frontend/
    ├── package.json, vite.config.ts, tsconfig.json, index.html
    ├── src/
    │   ├── main.tsx, App.tsx, index.css   # @import "tailwindcss";
    │   ├── api/ (client.ts, types.ts)     # fetch wrapper + CSRF header
    │   ├── auth/ (AuthProvider.tsx, RequireRole.tsx)  # UX only; backend enforces
    │   ├── components/ (Layout, Nav, LikertQuestion, StatCard, SyntheticBanner…)
    │   ├── pages/ (Login, Register, AcceptInvite, Dashboard*, SurveyList,
    │   │          SurveyTake, AdminUsers, AdminInvites, AdminDocuments,
    │   │          Analysis, Assistant, NotFound, Forbidden)
    │   └── test/
    └── e2e/ (Playwright specs)
```

C. Database schema

All tables use `id BIGINT GENERATED ALWAYS AS IDENTITY` (or UUID for public-facing IDs) plus `created_at TIMESTAMPTZ DEFAULT now()`. Extensions: `vector` and `citext`.

sql

```sql
-- Roles stored as enum: 'admin','manager','employee','customer'
branches(id, code VARCHAR(20) UNIQUE NOT NULL, name, city, region,
         is_synthetic BOOL NOT NULL DEFAULT false)

users(id, email CITEXT UNIQUE NOT NULL, password_hash TEXT NOT NULL,
      full_name VARCHAR(120) NOT NULL, role user_role NOT NULL,
      branch_id FK→branches NULL, is_active BOOL DEFAULT true,
      failed_login_count INT DEFAULT 0, locked_until TIMESTAMPTZ NULL,
      last_login_at TIMESTAMPTZ, is_synthetic BOOL DEFAULT false,
      CHECK (role IN ('admin','customer') OR branch_id IS NOT NULL))

invitations(id, token_hash CHAR(64) UNIQUE NOT NULL,
            role user_role NOT NULL CHECK (role IN ('manager','employee')),
            branch_id FK NOT NULL, email CITEXT NULL, created_by FK→users,
            expires_at TIMESTAMPTZ NOT NULL, used_at TIMESTAMPTZ NULL, used_by FK NU

sessions(id, user_id FK→users ON DELETE CASCADE, token_hash CHAR(64) UNIQUE NOT NULL
         csrf_token_hash CHAR(64) NOT NULL, created_at, last_seen_at,
         expires_at TIMESTAMPTZ NOT NULL, ip VARCHAR(45), user_agent VARCHAR(255))

audit_logs(id, actor_id FK NULL, action VARCHAR(60), target_type, target_id,
           metadata JSONB, ip, created_at)      -- never store passwords/tokens

constructs(id, code VARCHAR(10) UNIQUE, name, description)   -- TA, TS, EP, OE, DSQ
```

```
surveys(id, slug UNIQUE, title, target_role user_role NOT NULL
        CHECK (target_role IN ('employee','manager','customer')),
        version INT NOT NULL DEFAULT 1, status VARCHAR(10)
        CHECK (status IN ('draft','published','closed')),
        consent_text TEXT NOT NULL, UNIQUE(slug, version))

questions(id, survey_id FK ON DELETE CASCADE, construct_id FK NULL,
          code VARCHAR(20), text TEXT, qtype VARCHAR(15)
          CHECK (qtype IN ('likert5','single_choice','numeric','text')),
          options JSONB NULL, reverse_coded BOOL DEFAULT false,
          is_attention_check BOOL DEFAULT false, expected_value INT NULL,
          required BOOL DEFAULT true, position INT NOT NULL,
          UNIQUE(survey_id, code), UNIQUE(survey_id, position))

survey_responses(id, survey_id FK, user_id FK, branch_id FK NULL,
                 consent_given BOOL NOT NULL CHECK (consent_given),
                 submitted_at TIMESTAMPTZ, duration_seconds INT,
                 passed_attention BOOL, is_synthetic BOOL DEFAULT false,
                 UNIQUE(survey_id, user_id))           -- one response per survey

answers(id, response_id FK ON DELETE CASCADE, question_id FK,
        value_numeric NUMERIC NULL, value_text VARCHAR(1000) NULL,
        UNIQUE(response_id, question_id),
        CHECK (value_numeric IS NOT NULL OR value_text IS NOT NULL))

analysis_runs(id, created_by FK, analysis_type VARCHAR(30), params JSONB,
              data_scope VARCHAR(10) CHECK (data_scope IN ('real','synthetic','all')
              dataset_hash CHAR(64), n_used INT, n_excluded INT,
              results JSONB, warnings JSONB, library_versions JSONB, created_at)

documents(id, title, source_filename, version VARCHAR(20), sha256 CHAR(64) UNIQUE,
          classification VARCHAR(15) CHECK (classification IN
            ('public','customer','internal','confidential','restricted')),
          allowed_roles user_role[] NOT NULL, status VARCHAR(10)
          CHECK (status IN ('draft','published','archived')),
          uploaded_by FK, published_at, CHECK (cardinality(allowed_roles) > 0))

document_chunks(id, document_id FK ON DELETE CASCADE, chunk_index INT,
                section_heading TEXT, page_number INT NULL, content TEXT,
                token_count INT, embedding VECTOR(384) NOT NULL,
                tsv TSVECTOR GENERATED ALWAYS AS (to_tsvector('english', content)) S
                allowed_roles user_role[] NOT NULL,   -- denormalized copy for filte
                doc_status VARCHAR(10) NOT NULL,      -- denormalized copy
                UNIQUE(document_id, chunk_index))

assistant_queries(id, user_id FK, question_redacted TEXT, retrieved_chunk_ids BIGINT
                  cited_chunk_ids BIGINT[], refused BOOL, latency_ms INT, created_at
```

Relationships: a branch has many users and responses. A survey has many questions and responses. A response has many answers. A document has many chunks. Invitations are bound to one role and

one branch.

Indexes:

- `sessions(token_hash)` unique, `sessions(user_id)`, `sessions(expires_at)`
- `survey_responses(survey_id, branch_id)`, `answers(question_id)`
- `audit_logs(actor_id, created_at DESC)`
- `document_chunks USING hnsw (embedding vector_cosine_ops)`
- `document_chunks USING gin (tsv)`, `document_chunks USING gin (allowed_roles)`

Access-copy rule: whenever a document's `allowed_roles` or `status` changes, the same transaction must update every one of its chunks. A test enforces this.

D. API contract

All routes start with `/api/v1`. Errors always use one shape:

```json
{ "error": { "code": "FORBIDDEN", "message": "You do not have permission.", "detail
```

Method	Path	Permission	Purpose
GET	`/health`	public	Liveness + DB check
POST	`/auth/register`	public	Customer self-registration only
POST	`/auth/accept-invite`	public + valid invite	Employee/manager registration
POST	`/auth/login`	public	Creates session
POST	`/auth/logout`	authenticated	Deletes session
POST	`/auth/logout-all`	authenticated	Revokes all user sessions
GET	`/auth/me`	authenticated	Current user + permissions + CSRF token
POST	`/auth/change-password`	authenticated	Revokes other sessions

Method	Path	Permission	Purpose
GET/PATCH	`/users`, `/users/{id}`	`users:manage`	List, deactivate, change branch
POST/GET	`/invitations`	`invitations:create`	Create invite (returns token once)
GET	`/branches`	authenticated	Branch list for forms
GET	`/surveys`	`survey:respond`	Published surveys for *my* role
GET	`/surveys/{id}`	`survey:respond` + role match	Questions
POST	`/surveys/{id}/responses`	`survey:respond` + role match	Submit
GET	`/responses/me`	authenticated	My submissions
POST	`/admin/surveys`, PATCH `/admin/surveys/{id}`	`survey:manage`	Create/publish/close
POST	`/analysis/runs`	`analysis:run`	Run analysis
GET	`/analysis/runs`, `/analysis/runs/{id}`	`analysis:view`	Stored results
GET	`/analysis/export.csv`	`data:export`	De-identified dataset
GET	`/dashboards/research`	`dashboard:research`	Full overview
GET	`/dashboards/branch`	`dashboard:branch`	Manager: own branch, aggregated
POST/GET/PATCH	`/documents`	`documents:manage`	Upload, list, publish, reclassify
POST	`/assistant/query`	`assistant:use`	RAG Q&A

Examples

http

```
POST /api/v1/auth/register
{ "email": "asha@example.com", "password": "correct-horse-battery-9", "full_name": '
→ 201 { "id": 42, "email": "asha@example.com", "role": "customer" }
```

Any `role` field gets a `422` response, because the schema uses `extra="forbid"`.

```http
POST /api/v1/auth/login
{ "email": "asha@example.com", "password": "..." }
→ 200  Set-Cookie: bt_session=...; HttpOnly; Secure; SameSite=Lax; Path=/
{ "user": { "id": 42, "role": "customer", "full_name": "Asha K" },
  "csrf_token": "q1w2...", "permissions": ["survey:respond","assistant:use"] }
→ 401 { "error": { "code": "INVALID_CREDENTIALS", "message": "Email or password is
```

```http
POST /api/v1/surveys/3/responses        (header X-CSRF-Token: q1w2...)
{ "consent_given": true, "duration_seconds": 310,
  "answers": [ { "question_code": "TA1", "value": 4 }, { "question_code": "TA2R", "
→ 201 { "response_id": 918, "submitted_at": "2026-10-27T14:10:00Z" }
→ 409 { "error": { "code": "ALREADY_SUBMITTED", ... } }
→ 403 if survey.target_role != user.role
```

```http
POST /api/v1/analysis/runs
{ "analysis_type": "correlation", "data_scope": "synthetic",
  "variables": ["TA","EP","OE"], "method": "pearson", "level": "individual",
  "exclude_failed_attention": true, "p_adjust": "holm" }
→ 201 { "id": 17, "n_used": 212, "n_excluded": 9, "dataset_hash": "ab12...",
  "results": { "pairs": [ { "x": "TA", "y": "EP", "r": <computed>, "ci95": [<lo>, <h
      "p_raw": <computed>, "p_adj": <computed>, "n": 212 } ] },
  "warnings": ["Correlation does not establish causation.", "Data scope: SYNTHETIC.'
```

Every number comes from the engine at runtime. Nothing is hard-coded.

```http
```

```
POST /api/v1/assistant/query
{ "question": "How do I reset my net banking password?" }
→ 200 { "answer": "You can reset it via the 'Forgot password' link... [1]",
  "citations": [ { "n": 1, "document_title": "Digital Banking FAQ", "version": "1.2'
      "section": "Password reset", "page": null, "chunk_id": 551, "snippet": "..." }
  "refused": false, "mode": "extractive" }
```

E. Authentication, session and authorization design

Registration

- Public `/auth/register` only ever creates a **customer**. The request schema has no role field, and unknown fields are rejected.
- Employees and managers register through `/auth/accept-invite`. The admin creates a single-use invite bound to a role and branch, with a 72 h expiry. The token is shown once, and only its SHA-256 hash is stored.
- The **admin** account is created only from the CLI: `uv run python -m app.cli create-admin --email ...`. The CLI prompts for the password. No API endpoint can create or promote to admin, and a test checks this.

Passwords

- Argon2id via `argon2-cffi`.
- 12–128 characters, checked against a bundled common-password list.
- Rehash on login if the parameters have changed.

Sessions

- Token: `secrets.token_urlsafe(32)`. The cookie `bt_session` is `HttpOnly`, `Secure` in production, `SameSite=Lax`, `Path=/`.
- The DB stores `sha256(token)`, so a database leak doesn't expose usable sessions.
- Idle timeout is 30 min (sliding, with `last_seen_at` updated at most every 5 min). Absolute lifetime is 8 h.
- A new token is issued on every login (prevents session fixation). Logout deletes the row. Password change deletes all other sessions.

CSRF

- Synchronizer token: `/auth/login` and `/auth/me` return a `csrf_token`, and its hash is stored on the session row.

- The frontend keeps it in memory and sends `X-CSRF-Token` on POST, PATCH and DELETE. The backend compares in constant time.

Brute force

- `slowapi` limits: 5 login attempts/min per IP, plus a per-account lock of 15 min after 10 failures.
- Login failures always return the same generic message.

Authorization (always on the backend)

- `get_current_user` reads the cookie, looks up the hash, and checks expiry and `is_active`.
- `require_permission("analysis:run")` checks the `ROLE_PERMISSIONS` dict in `core/permissions.py`.
- **Object-level checks** run inside services: survey role match, manager restricted to `user.branch_id`, users seeing only their own responses.
- **k-anonymity:** any aggregate shown to a manager with a group size under `MIN_GROUP_SIZE=5` is suppressed and returned as `"suppressed": true`.

The frontend `RequireRole` only hides UI. Every protected route has a backend test that calls the API directly with the wrong role and expects 403.

F. Role-permission matrix

Permission	Admin/Researcher	Manager	Employee	Customer
`survey:respond` (own-role surveys)	–	✓	✓	✓
`responses:view_own`	–	✓	✓	✓
`survey:manage`	✓	–	–	–
`dashboard:research` (all data)	✓	–	–	–
`dashboard:branch` (own branch, aggregated, k≥5)	✓	✓	–	–
`analysis:run` / `analysis:view`	✓	–	–	–
`data:export` (de-identified)	✓	–	–	–
`users:manage`	✓	–	–	–

Permission	Admin/Researcher	Manager	Employee	Customer
`invitations:create`	✓	–	–	–
`documents:manage`	✓	–	–	–
`assistant:use`	✓	✓	✓	✓
RAG docs visible	all published	public, customer, internal, confidential	public, customer, internal	public, customer
`audit:view`	✓	–	–	–

Admin doesn't respond to surveys, which keeps the researcher's own answers out of the data. Individual responses are never visible to managers.

G. Survey design and statistical formulas

Research model (associational, not causal)

Code	Construct	Measured in
TA	Technology Adoption & Use	Employee, Manager
TS	Training & Technical Support	Employee
EP	Employee Performance (self-reported)	Employee
OE	Organisational Effectiveness	Manager (+ employee perception)
DSQ	Digital Service Quality	Customer
CS	Customer Satisfaction	Customer
TR	Trust & Perceived Security	Customer

Hypotheses (all worded as associations):

- **H1:** TA is positively associated with EP.
- **H2:** TS is positively associated with EP.
- **H3:** TA is positively associated with OE.
- **H4:** At branch level, mean DSQ is associated with mean CS.
- **H5:** TA and TS together explain variance in EP (OLS).

Instrument rules

- 3–5 Likert items per construct (1 = Strongly disagree … 5 = Strongly agree), at least 1 reverse-coded item, and 1 attention check per survey.
- Demographics use bands only (age band, tenure band, role level).
- A consent screen comes first. No names, no account numbers, no financial data in any survey.
- Write your own items. If you adapt items from published scales such as TAM or SERVQUAL, cite them in `research-methodology.md`.

Data-cleaning policy (set before analysis and recorded in every `analysis_runs` row):

- Exclude responses that fail the attention check.
- Construct score is computed only if at least 70% of its items are answered.
- Otherwise, listwise deletion per analysis.

Formulas

Reverse coding (5-point scale): $x' = (\min + \max) - x = 6 - x$

Construct score for respondent i over k answered items: $\bar{C}_i = \frac{1}{k}\sum_{j=1}^{k} x'_{ij}$ *Assumption:* averaged Likert items are treated as approximately interval data. As a robustness check, Spearman is reported alongside Pearson.

Descriptives: $\bar{x}$, $s = \sqrt{\frac{\sum(x_i-\bar{x})^2}{n-1}}$, median, IQR, n.

Cronbach's alpha (k items, σ_j^2 item variance, σ_T^2 variance of total score): $\alpha = \frac{k}{k-1}\left(1 - \frac{\sum_{j=1}^{k}\sigma_j^2}{\sigma_T^2}\right)$

Report alpha as computed and interpret it with conventional bands (≥ 0.70 acceptable). Never tune items to reach a target.

Pearson correlation and significance: $r = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum(x_i-\bar{x})^2\sum(y_i-\bar{y})^2}}, \qquad t = \frac{r\sqrt{n-2}}{\sqrt{1-r^2}},\ df = n-2$

95% CI (Fisher z): $z = \operatorname{atanh}(r),\ SE = 1/\sqrt{n-3},\ CI = \tanh(z \pm 1.96\,SE)$

Spearman ρ: Pearson computed on ranks.

Multiple testing: Holm–Bonferroni across the hypothesis family. Report both raw and adjusted p-values.

OLS regression (statsmodels, HC3 robust standard errors): $\hat{\beta} = (X^\top X)^{-1}X^\top y, \quad R^2 = 1 - \frac{SS_{res}}{SS_{tot}}, \quad R^2_{adj} = 1 - (1-R^2)\frac{n-1}{n-p-1}$ Also report standardized betas, and $VIF_j = 1/(1-R_j^2)$ with a warning if any VIF exceeds 5. Include residual diagnostics.

Group comparisons:

- Welch's t: $t = \frac{\bar{x}_1-\bar{x}_2}{\sqrt{s_1^2/n_1 + s_2^2/n_2}}$
- Cohen's d: $\frac{\bar{x}_1-\bar{x}_2}{s_p}$
- One-way ANOVA: $F = MS_B/MS_W$, with $\eta^2 = SS_B/SS_T$
- Kruskal–Wallis as the non-parametric alternative

Branch-level analysis (H4): aggregate the means per branch first, with a minimum of 5 responses per branch. Flag the **ecological-fallacy** limitation, and note that employees nested in branches violate independence. A stretch option is `statsmodels` MixedLM with a branch random intercept.

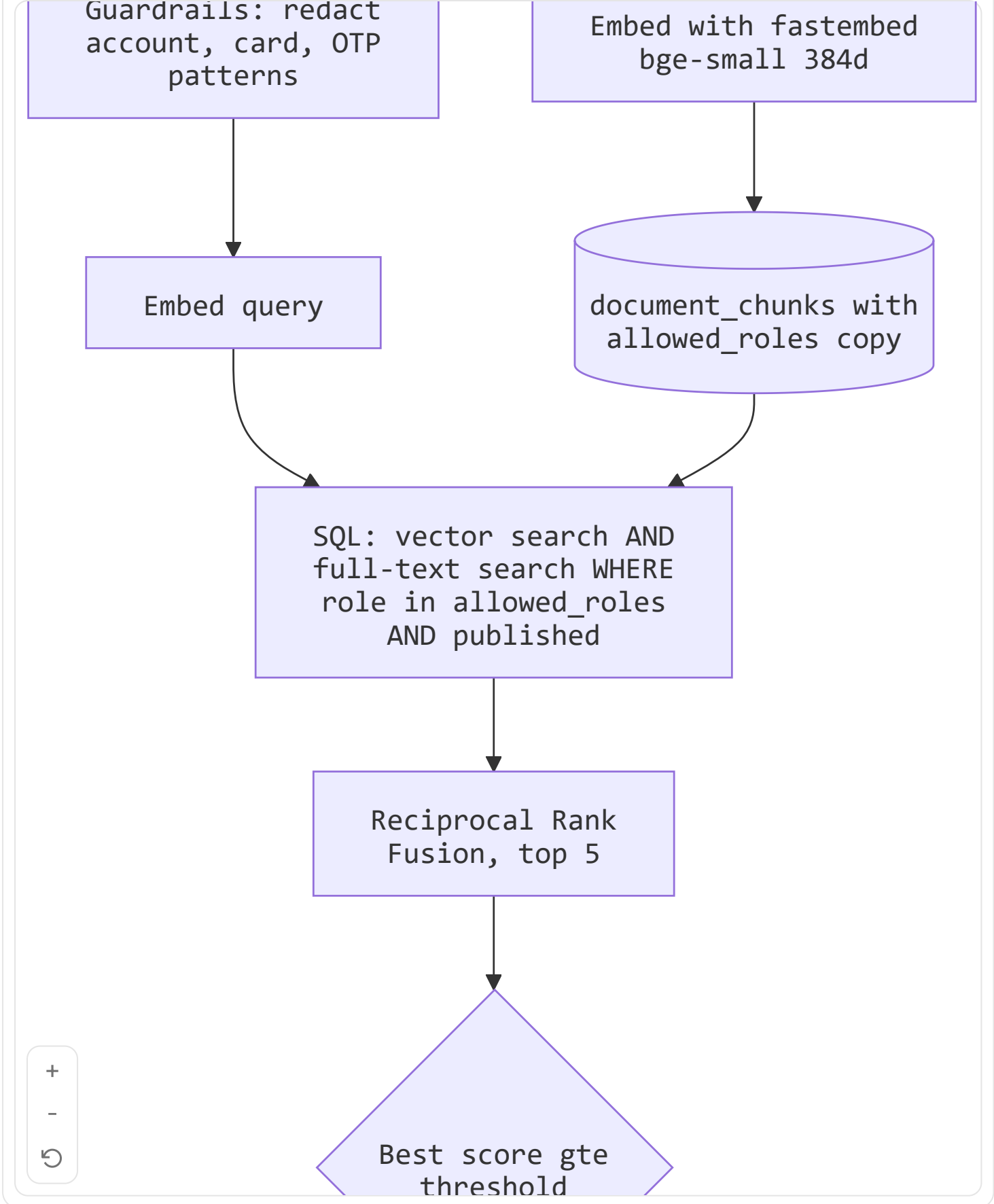
Sample size guidance: detecting $r = 0.3$ at $\alpha = .05$ with 80% power needs about 85 respondents. The UI shows a warning when n falls below the threshold you configure.

Integrity safeguards (built into the code):

- Every result is computed from the database at run time and stored with `dataset_hash`, parameters and library versions.
- Results from synthetic data always show a red **SYNTHETIC DATA – NOT RESEARCH FINDINGS** banner. `data_scope` is never mixed silently.
- The result-text templates use only "associated with", "correlated with" and "explained variance". A unit test fails if the text contains "causes", "leads to", "drives" or "impact of".
- Non-significant results are displayed just like significant ones.

H. RAG architecture

mermaid



Document metadata: title, filename, version, sha256, classification, `allowed_roles`, status, uploader, published_at. Chunk metadata: heading, page, chunk_index, token_count.

Corpus: write your own synthetic FAQ documents for a fictional bank (digital banking, card blocking, UPI safety, account opening, an internal IT policy, a confidential branch-targets memo, and a restricted audit note). This avoids copyright problems and gives you a document at every access level to test against.

Chunking

- Split Markdown on headings. Split PDFs per page, then by paragraph.
- Target about 200–300 tokens (roughly 1,000 characters) with about 15% overlap.
- Never cross a heading boundary, and keep the heading as a prefix for context.

Retrieval

- Run the vector query (`embedding <=> :q`) and the full-text query (`ts_rank`) separately, each limited to the top 20.
- Fuse them with RRF: $\text{score}(d) = \sum_q \frac{1}{60+\text{rank}_q(d)}$
- Keep the top 5. Refuse if the best cosine similarity is below `RAG_MIN_SIMILARITY` (tune it on your golden set).

Access control (most important)

- The filter `allowed_roles && ARRAY[:role]::user_role[] AND doc_status='published'` sits **inside the SQL WHERE clause** of both queries, before ranking.
- Restricted text never reaches the LLM, the logs or the response. Role comes only from the server-side session, never from the request body.

Citations

- Context passages are numbered… The prompt requires citing by number.
- The validator removes any number not in the retrieved set. If no valid citation remains, the answer becomes the refusal message.
- In `extractive` mode the answer is the top passages, quoted with their citations.

Guardrails

- Redact patterns like 9–18 digit account numbers, Luhn-valid card numbers, OTP/PIN/CVV phrases, and IFSC+account combinations. Remind the user never to share them.
- Document text is wrapped as untrusted data, to defend against prompt injection.
- Show a "General information, not financial advice" footer.
- Log only the redacted question.

RAG evaluation: keep 20–30 golden Q&A pairs, each with the expected source document. Report hit@5 and the refusal rate on out-of-scope questions.

I. Dependencies

These ranges assume you install current releases and lock them. Recent upstream releases include React 19.3 (Sep 2026), Tailwind v4, and FastAPI, which ships frequent minor releases. Run the install, then commit `uv.lock` and `package-lock.json`, and treat the lockfiles as the source of truth.

Runtime: Python 3.12, Node 22 or 24 LTS, PostgreSQL 17 (`pgvector/pgvector:pg17` Docker image), Docker + Compose v2.

Backend (`pyproject.toml`)	Range
fastapi[standard]	latest stable at install (pin in lockfile)
pydantic / pydantic-settings	`>=2.7,<3` / `>=2.3,<3`
sqlalchemy	`>=2.0.30,<2.1`
alembic	`>=1.13,<2`
psycopg[binary]	`>=3.2,<4`
pgvector	`>=0.3,<1`
argon2-cffi	`>=23.1`
slowapi	`>=0.1.9`
pandas / numpy / scipy / statsmodels	`>=2.2,<3` / `>=2,<3` / `>=1.13` / `>=0.14`
fastembed	`>=0.4`
pypdf	`>=5`
httpx (LLM calls + tests)	`>=0.27`
Faker	`>=25`
Dev: pytest, pytest-cov, ruff, mypy	`>=8` / `>=5` / latest / latest

Frontend (`package.json`)	Range
react, react-dom	`^19`
typescript	`^5`
vite, @vitejs/plugin-react	latest major at install
tailwindcss, @tailwindcss/vite	`^4` (CSS-first config, no `tailwind.config.js` needed)
react-router	`^7`
@tanstack/react-query	`^5`

Frontend (`package.json`)	Range
react-hook-form, zod, @hookform/resolvers	`^7` , latest, latest
recharts	latest major
Dev: vitest, @testing-library/react, @testing-library/user-event, jsdom, @playwright/test, eslint	latest

J + K. Phase-by-phase plan with acceptance criteria and tests

Each phase ends with a working app, green tests, and an entry in `docs/handoff-log.md` .

Phase 0: Scaffold and health check

- **Files:** root `README.md` , `.gitignore` , `.env.example` , `docker-compose.yml` ; `backend/pyproject.toml` , `app/main.py` , `app/core/config.py` , `app/db/session.py` , `app/api/routes/health.py` , `tests/test_health.py` ; `frontend/` from the Vite template, plus `vite.config.ts` (proxy `/api` to `localhost:8000` , Tailwind plugin), `src/index.css` , `src/App.tsx` .
- **Prerequisites:** Git, Docker, Python 3.12, uv, Node LTS.
- **Steps:** create the repo → start Postgres in Compose → FastAPI app factory → `/api/v1/health` runs `SELECT 1` → React page shows the health status.
- **Commands:**

```bash
docker compose up -d db
cd backend && uv sync && uv run fastapi dev app/main.py
cd frontend && npm install && npm run dev
uv run pytest -q ; npm run test
```

- **Acceptance:** `GET /api/v1/health` returns `{"status":"ok","db":"ok"}` . The browser at :5173 shows "API: ok". `/api/docs` loads.
- **Tests:** health returns 200; health with the DB stopped returns 503; a frontend render test passes.
- **Common errors:**
  - Port 5432 in use: stop your local Postgres or change the port mapping.
  - `connection refused` : the DB container isn't healthy yet. Add a healthcheck.
  - Tailwind styles missing: the CSS file needs `@import "tailwindcss";` .

Phase 1: Database models and migrations

- **Files:** `app/db/base.py`, everything in `app/models/`, `alembic/env.py`, first migration (`CREATE EXTENSION vector, citext`), `app/cli.py` (`seed-branches`), `tests/conftest.py` (separate test DB, transaction rollback per test), `tests/integration/test_constraints.py`.
- **Prerequisites:** Phase 0.
- **Steps:** define the models → autogenerate the migration → **review it by hand** (enums, the CHECK constraints, the HNSW index, the generated tsvector column usually need manual edits) → upgrade → seed branches.
- **Commands:** `uv run alembic revision --autogenerate -m "initial"`, `uv run alembic upgrade head`, `uv run python -m app.cli seed-branches`
- **Acceptance:** `alembic downgrade base && alembic upgrade head` runs cleanly. All constraints exist.
- **Tests:** a duplicate email raises IntegrityError; an employee with no branch fails the CHECK; a response with `consent_given=false` fails; a duplicate response per survey/user fails.
- **Common errors:**
  - `type "vector" does not exist`: the extension isn't created, or you're not using the pgvector image.
  - Autogenerate misses enums or CHECK constraints: add them manually.
  - Tests polluting the dev DB: check that `TEST_DATABASE_URL` is set.

Phase 2: Authentication

- **Files:** `core/security.py`, `core/csrf.py`, `core/rate_limit.py`, `core/errors.py`, `services/auth_service.py`, `schemas/auth.py`, `api/deps.py`, `api/routes/auth.py`, `cli.py` (`create-admin`); frontend `api/client.ts`, `auth/AuthProvider.tsx`, `pages/Login.tsx`, `Register.tsx`, `AcceptInvite.tsx`; `tests/security/test_auth.py`.
- **Steps:** hashing → session create/lookup/expire → register (customer only) → login/logout/me → CSRF check → rate limit → `create-admin` CLI → frontend forms with zod validation.
- **Acceptance:** you can register, log in, refresh the page and stay logged in, and log out. The cookie is HttpOnly. Admin can only be created by the CLI.
- **Tests:**
  - Register with a `role` field returns 422, and with `role:"admin"` also 422.
  - Wrong password returns 401 with a generic message.
  - The 6th login attempt within a minute returns 429.
  - The DB contains only the token hash, never the raw token.

    - Logout invalidates the cookie (`/me` afterwards returns 401).
    - An expired session returns 401.
    - POST without the CSRF header returns 403.
    - Password hashes start with `$argon2id$` .
- **Common errors:**
    - Cookie not set in dev: you called `:8000` directly instead of using the `/api` proxy.
    - `Secure` cookie dropped over http: set `COOKIE_SECURE=false` in dev only.
    - 403 on every POST: the client isn't sending `X-CSRF-Token` .

Phase 3: RBAC, invitations, admin user management, audit log

- **Files:** `core/permissions.py` , `api/routes/users.py` , `invitations.py` , `branches.py` , `models/audit.py` usage; frontend `RequireRole.tsx` , `AdminUsers.tsx` , `AdminInvites.tsx` , `Forbidden.tsx` ; `tests/security/test_rbac.py` .
- **Steps:** permission dict → `require_permission` dependency → invite create and accept flow → user deactivation (which revokes sessions) → audit entries for login, logout, invite and role changes.
- **Acceptance:** each role sees the right nav, and a direct API call with the wrong role returns 403.
- **Tests:**
    - A parametrized test over **every route × every role** checks that the status matches the matrix.
    - An invite can't be reused, can't be used after expiry, and can't create an admin.
    - A deactivated user's sessions are revoked immediately.
    - No endpoint can change role to admin.
- **Common errors:**
    - Route forgotten in the RBAC test: generate the route list from `app.routes` so new routes fail the test until they're classified.

Phase 4: Survey engine

- **Files:** `seed/surveys/{employee,manager,customer}.json` , `services/survey_service.py` , `schemas/survey.py` , `routes/surveys.py` , `routes/responses.py` ; frontend `SurveyList.tsx` , `SurveyTake.tsx` , `components/LikertQuestion.tsx` , consent step; tests.
- **Steps:** seed the instruments → list surveys by role → render questions → validate answers server-side (range 1–5, required items, question belongs to the survey, attention check) → submit inside

a transaction.

- **Acceptance:** each role completes its own survey once. A second attempt is blocked. Responses are stored with the branch.
- **Tests:**
  - An employee can't fetch or submit the customer survey (403).
  - A value of 6 or 0 returns 422; a missing required item returns 422.
  - Resubmitting returns 409.
  - Submitting to a draft or closed survey returns 404 or 409.
  - Reverse-coded flags round-trip correctly.
- **Common errors:**
  - Question codes mismatch between the JSON and the frontend: always submit by `question_code`.
  - Partial saves: keep the response and its answers in a single transaction.

Phase 5: Synthetic data generator

- **Files:** `services/synthetic/generator.py`, `cli.py` (`generate-synthetic --seed 42 --branches 20 --employees 300 --managers 40 --customers 400`, `purge-synthetic`), tests.
- **Steps:** generate latent construct scores with a *configurable* correlation matrix → convert to Likert items (with noise, rounding and clipping) → add about 5% attention failures and missing values → set `is_synthetic=true` everywhere.
- **Acceptance:** the generator is deterministic for a given seed, and purging removes only synthetic rows.
- **Tests:** the same seed gives the same dataset hash; **known-effect recovery** (plant ρ = 0.40 and check the engine estimates within ±0.1 at large n). This validates the pipeline, not the hypotheses. Purging leaves real data intact.
- **Common errors:**
  - Clipping squashes the variance: tune the latent scale and print the descriptives.

Phase 6: Statistical analysis engine

- **Files:** `services/stats/*.py`, `routes/analysis.py`, `schemas/analysis.py`, `tests/unit/test_stats_*.py`, `docs/research-methodology.md`.
- **Steps:** build the long→wide DataFrame from the DB → apply reverse coding, exclusions and scoring → descriptives → alpha → correlations (with CI and Holm) → OLS + VIF → group tests → store `analysis_runs` → generate the cautious text summary.

- **Acceptance:** the admin can run every analysis type, re-running identical data gives the same `dataset_hash` and the same results, and n_used and n_excluded are both shown.
- **Tests:**
  - **Hand fixture:** for $x =$ and $y =$, $r = 6/\sqrt{60} \approx 0.7746$.
  - Our alpha, Pearson, Welch and OLS match scipy/statsmodels to 10^{-8}.
  - Reverse coding maps 1→5.
  - n below the minimum gives an `insufficient_data` warning instead of a number.
  - The causal-language linter passes.
  - A manager calling `/analysis/runs` gets 403.
- **Common errors:**
  - NaN results: a zero-variance column (all respondents gave the same answer). Detect it and warn.
  - Decimal vs float types from Postgres NUMERIC: cast to float in pandas.

Phase 7: Dashboards

- **Files:** `routes/dashboards.py`; frontend `DashboardResearch.tsx`, `DashboardBranch.tsx`, `DashboardEmployee.tsx`, `DashboardCustomer.tsx`, `Analysis.tsx`, `components/{StatCard, CorrelationHeatmap, ConstructBarChart, SyntheticBanner}.tsx`.
- **Steps:** research dashboard (response counts, construct means with SD, alpha table, correlation heatmap, regression table, filters for data scope and branch) → manager branch view (aggregated, k ≥ 5) → employee/customer pages (survey status, my submissions, assistant link).
- **Acceptance:** the charts render from API data, the synthetic banner shows whenever scope includes synthetic data, and small groups show "suppressed".
- **Tests:** manager branch view only returns their branch even if `?branch_id=` is forged; groups under 5 are suppressed; component tests for the banner and the empty states.
- **Common errors:**
  - Recharts renders blank: the parent needs an explicit height.
  - Stale data after logout: call `queryClient.clear()` on logout.

Phase 8: RAG assistant

- **Files:** `seed/faq_docs/*.md` (one per classification), `services/rag/*.py`, `routes/documents.py`, `routes/assistant.py`, `cli.py` (`ingest-docs`); frontend `AdminDocuments.tsx`, `Assistant.tsx` (answer + citation cards); `tests/rag/*`.

- **Steps:** loader → chunker → embedder (cache the model directory) → upsert with access copies → hybrid retriever with the SQL role filter → guardrails → LLM adapter (`none` first) → citation validator → query log.
- **Acceptance:** in-scope questions are answered with valid citations, out-of-scope questions are refused, and it works with `LLM_PROVIDER=none` .
- **Tests (security-critical):**
    - For every role, ask a question whose exact answer is only in the restricted or confidential document. Assert that no retrieved chunk ID belongs to a disallowed document, and that the snippet text is absent from the response.
    - After reclassifying a doc to `restricted` , an employee immediately loses access.
    - Draft documents are never retrieved.
    - A Luhn-valid card number in the question is redacted in the stored log.
    - A citation number outside the retrieved set is stripped.
    - Golden set hit@5 meets your target (record whatever value you actually measure).
- **Common errors:**
    - Dimension mismatch: the model output size must equal `VECTOR(384)` .
    - First run is slow: the model downloads once. Set `FASTEMBED_CACHE_DIR` .
    - HNSW returns nothing with a strict filter: increase `hnsw.ef_search` or use exact search (it's fine for a small corpus).

Phase 9: Hardening, E2E, docs, deployment

- **Files:** `e2e/*.spec.ts` , `backend/Dockerfile` , `frontend/Dockerfile` (build stage), `Caddyfile` , Compose `prod` profile, security headers middleware, all of `docs/*` .
- **Steps:** Playwright flows → headers (CSP, `X-Content-Type-Options` , `Referrer-Policy` , HSTS in prod) → `/api/docs` disabled in prod → backup script → write the docs.
- **Acceptance:** a fresh clone runs with `docker compose --profile prod up` to a working HTTPS app. All test suites pass, and backend coverage is ≥ 80% (≥ 95% for auth, permissions and RAG access).
- **E2E tests:**
    - Customer: register → survey → assistant.
    - Invited employee: accept invite → survey.
    - Manager: branch dashboard.
    - Admin: run analysis → export.

    - Visiting `/admin` as a customer shows Forbidden, and the API returns 403.

- **Common errors:**
    - SPA routes give 404 on refresh: add Caddy `try_files {path} /index.html`.
    - Cookies lost behind the proxy: set FastAPI `--proxy-headers` and trust the forwarded IPs.

L. Deployment architecture and environment variables

Target: a single small Linux VM (or any Docker host) running Docker Compose with `caddy` (automatic HTTPS; serves the static React build; proxies `/api` to `backend:8000`), `backend` (uvicorn, 2–4 workers, non-root user), and `db` (pgvector image on a named volume, **no public port**). Run a nightly `pg_dump` to off-host storage.

Variable	Example / note
`APP_ENV`	`development` / `production`
`DATABASE_URL`	`postgresql+psycopg://bt:***@db:5432/banktech`
`TEST_DATABASE_URL`	separate DB, tests only
`POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB`	strong random password
`SECRET_KEY`	64+ random characters (`python -c "import secrets;print(secrets.token_urlsafe(64))"`)
`COOKIE_SECURE`	`true` in production
`SESSION_IDLE_MINUTES` / `SESSION_ABSOLUTE_HOURS`	`30` / `8`
`LOGIN_RATE_LIMIT`	`5/minute`
`MIN_GROUP_SIZE`	`5`
`MIN_ANALYSIS_N`	e.g. `30`
`ALLOWED_HOSTS`	`banktech.example.com`
`EMBEDDING_MODEL` / `FASTEMBED_CACHE_DIR`	`BAAI/bge-small-en-v1.5` / `/models`

Variable	Example / note
`RAG_TOP_K` / `RAG_MIN_SIMILARITY`	`5` / tuned value
`LLM_PROVIDER`	`none` | `openai` | `gemini` | `ollama`
`LLM_API_KEY` / `LLM_MODEL` / `LLM_BASE_URL`	only if the provider is not `none`
`MAX_UPLOAD_MB`	`10`
`DOMAIN`	used by Caddy
`ENABLE_API_DOCS`	`false` in production

Checklist:

- `.env` is in `.gitignore` and only `.env.example` is committed.
- The app refuses to start if `APP_ENV=production` and `SECRET_KEY` is the default or `COOKIE_SECURE` is false.

M. Riskiest technical decisions and how to resolve them

#	Risk	Resolution
1	**Unit-of-analysis mismatch:** employee, manager and customer responses can't be linked one-to-one	Test H1–H3 at the individual level within each survey. Test cross-group hypotheses (H4) only at branch level. Generate at least 15–20 branches, and document the ecological fallacy.
2	**RAG access leaks** through stale denormalized chunk metadata	Update the chunk copies in the same transaction as the document, filter in SQL before ranking, and run per-role leak tests in CI.
3	**Synthetic results mistaken for findings**	The `data_scope` field, mandatory banner, `is_synthetic` columns, and a methodology note: synthetic data validates the pipeline only.
4	**Cookie/CSRF misconfiguration** across dev and prod	Same-origin proxy in both environments, a startup config validator, and E2E login through the real proxy.
5	**Alembic autogenerate gaps** (enums, CHECK constraints, vector index, generated column)	Review every migration by hand, and run a downgrade/upgrade round-trip test in CI.
6	**Statistical misuse** (p-hacking, causal wording, ignoring	Pre-registered hypotheses in the docs, Holm correction, effect sizes with CIs, the causal-language linter, and reporting non-

#	Risk	Resolution
	assumptions)	significant results.
7	**Embedding model download and size** in CI and Docker	Use fastembed (ONNX, no torch), cache the model in a volume or bake it into the image, and allow a deterministic fake embedder for unit tests.
8	**Respondent privacy:** managers inferring individuals	Aggregates only, k ≥ 5 suppression, banded demographics, and exports with no user IDs.
9	**Admin bootstrap mistakes**	CLI-only creation, no promotion endpoint, and an RBAC test that scans all routes.
10	**Token-limited IDE causing drift** between design and code	One module per request, each with its file list and tests. Antigravity runs `pytest` and `npm test` after applying the files and appends results to `handoff-log.md`. Paste failures back to me with the full traceback.

How to request modules

A good request looks like this: *"Phase 2, backend part: generate* `core/security.py`, `core/csrf.py`, `services/auth_service.py`, `schemas/auth.py`, `api/deps.py`, `routes/auth.py` *and* `tests/security/test_auth.py`. *Current state: Phase 1 is done and all tests pass."*

When you send a request like that, I'll reply with the full files, the exact commands, and a short Antigravity instruction block you can paste in to apply them and run the tests.